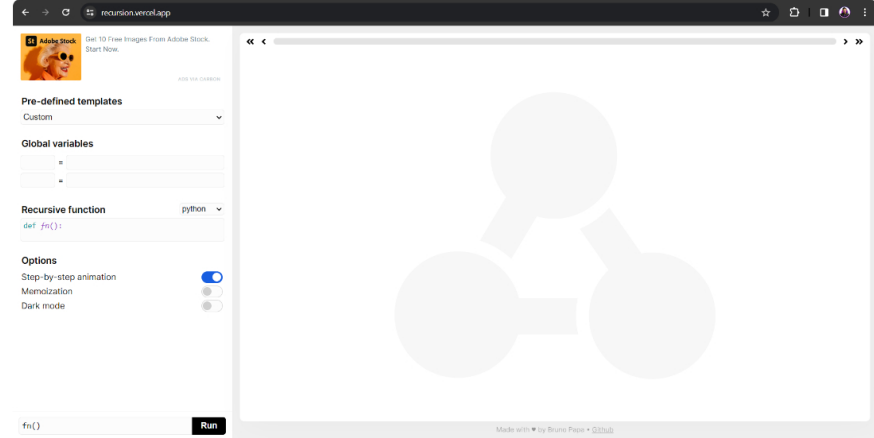


- মডিউল ১০ঃ Disjoint Set Union >
- মডিউল ১১ঃ DSU Cycle Detection এবং MST >
- মডিউল ১৩ঃ বেসিক গ্রাফ রিক্যাপ >
- মডিউল ১৪ঃ প্রব্লেম সল্ভিং গ্রাফ অ্যালগরিদম দিয়ে >
- মডিউল ১৫ঃ প্রব্লেম সল্ভিং ২ গ্রাফ অ্যালগরিদম দিয়ে >
- মডিউল ১৬ঃ বেসিক ডাইনামিক প্রোগ্রামিং >
- মডিউল ১৭-০১ঃ ইনট্রাকশন
- মডিউল ১৭-০২ঃ ডাইনামিক প্রোগ্রামিং কি?
- মডিউল ১৭-০৩ঃ ফ্যাক্টোরিয়াল নাম্বার
- মডিউল ১৭-০৪ঃ Fibonacci Series
- মডিউল ১৭-০৫ঃ Fibonacci Subproblem
- মডিউল ১৭-০৬ঃ Fibonacci Memoization
- মডিউল ১৭-০৭ঃ Bottom up approach in Fibonacci series
- মডিউল ১৮ঃ Knapsack >
- মডিউল ১৮-০১ঃ ইনট্রাকশন
- মডিউল ১৮-০২ঃ Knapsack

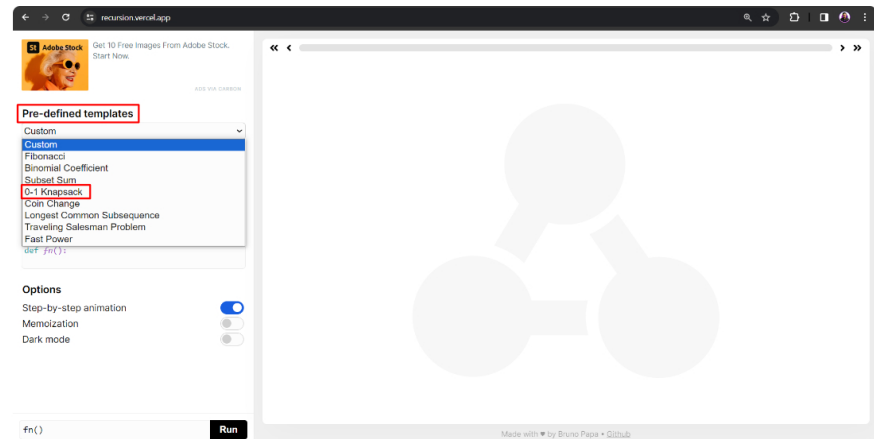
মডিউল ১৮-০২ঃ Knapsack Recursive Approach

প্রথমে নিচের ওয়েবসাইটে যাবেন,

<https://recursion.vercel.app/>

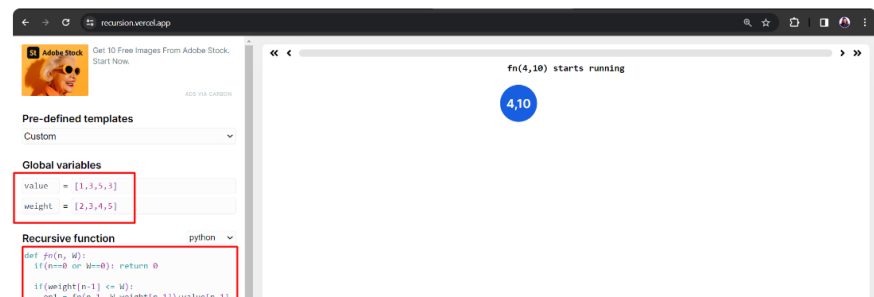


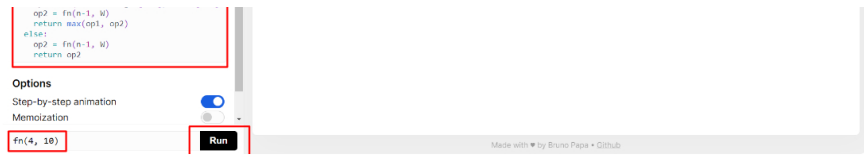
এরপর Pre-defined templates এর মধ্যে 0-1 Knapsack কে choose করবেন।



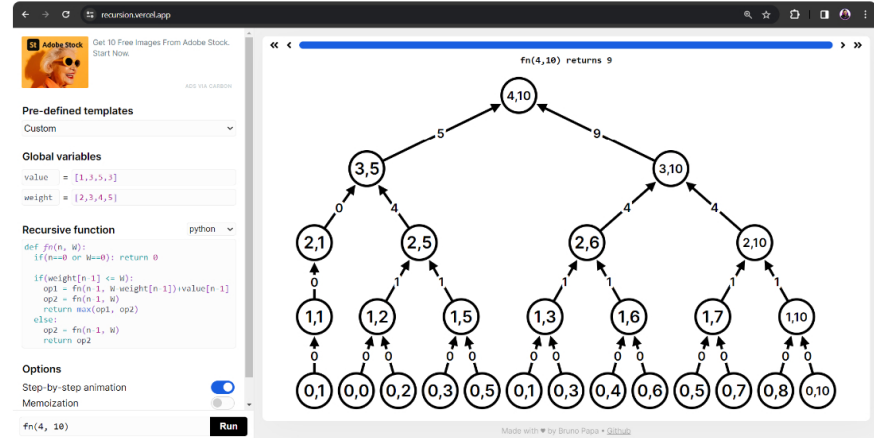
এরপর নিচের কোডটি দিয়ে দিবেন এবং Global variables , function call এ মানগুলো সেট করে দিবেন।

```
1 def fn(n, w):
2
3     if(n==0 or w==0): return 0
4
5
6
7     if(weight[n-1] <= w):
8
9         op1 = fn(n-1, w-weight[n-1])+value[n-1]
10
11        op2 = fn(n-1, w)
12
13        return max(op1, op2)
14
15    else:
16
17        op2 = fn(n-1, w)
18
19    return op2
```





Run button এ ক্লিক করলে visualization টি show করবে।



এইভাবে আপনি নিজেই 0-1 Knapsack এর recursion এর approach টিকে visualization করতে পারবে।

সম্পূর্ণ কোডটি

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int knapsack(int n, int weight[], int value[], int W)
6
7 {
8
9     if (n == 0 || W == 0)
10
11         return 0;
12
13     if (weight[n - 1] <= W)
14
15     {
16
17         // duita option
18
19         // niye dekhbo, na niye dekhbo
20
21         int op1 = knapsack(n - 1, weight, value, W - weight[n - 1]) + value[n - 1];
22
23         int op2 = knapsack(n - 1, weight, value, W);
24
25         return max(op1, op2);
26
27     }
28
29     else
30
31     {
32
33         // ekta option
34
35         // na niyei dekhte hobe
36
37         int op2 = knapsack(n - 1, weight, value, W);
38
39         return op2;
40
41     }
42
43 }
44
45 int main()
46
47 {
48
49     int n;
50
51     cin >> n;
52
53     int weight[n], value[n];
54
55     for (int i = 0; i < n; i++)
56
57     {
58
59         cin >> weight[i];
60
61     }
62
63     for (int i = 0; i < n; i++)
64
65     {
```

```
66
67     cin >> value[i];
68
69 }
70
71 int W;
72
73 cin >> W;
74
75 cout << knapsack(n, weight, value, W) << endl;
76
77 return 0;
78
79 }
```

Time Complexity: $O(2^n)$

<

Previous

সিডিউল ১৮-১৪ Knapsack

Next

সিডিউল ১৮-৩-৪৪ Knapsack Top Down Approach
ইমপ্লিমেন্টেশনসহ

>

Last updated September 9, 2024

